Tuning Hashes

Selected Projects in Performance Engineering of
Hash-Based Data Structures and Hash Functions

Calvin James Higgins
Department of Computer Science and Statistics
University of Rhode Island
March 15, 2026

A portfolio presented for the degree of
Master of Science in Computer Science

1 Introduction

I extract and tune parameters to improve performance. I selected three projects to illustrate this single theme .

TODO: Additionally, the student will add a single overall introduction (300-words or more) that motivates why they selected those three projects as exemplary of their accomplishments in the program.

2 Non-Uniform Binary Fuse Filters

This project was completed for CSC 592: Algorithms for Big Data.

2.1 Motivation

Distributed systems often require fast and small set data structures. For example, content delivery networks avoid caching one-hit wonders by querying a visited set [3]. Assuming $n \ll |U|$, such exact membership queries take $\Omega(\lg |U|)$ bits per key, which is impractical for large U. By permitting false positive results with some probability ϵ, the lower bound is reduced to $\lg \frac{1}{\epsilon}$ bits per key [1]. In other words, *approximate* membership queries can be much more memory efficient than exact membership queries.

A binary fuse filter [2] is a recent static probabilistic data structure for approximate membership queries. Membership information is stored in an array of m-bit values partitioned into disjoint segments. To query the filter, a key is hashed to a fingerprint value and to three array entries located in consecutive segments (see Figure 1). If the bitwise-XOR of the entries is equal to the fingerprint, then the query *might* be in the set. Otherwise, the query *definitely* is not in the set.

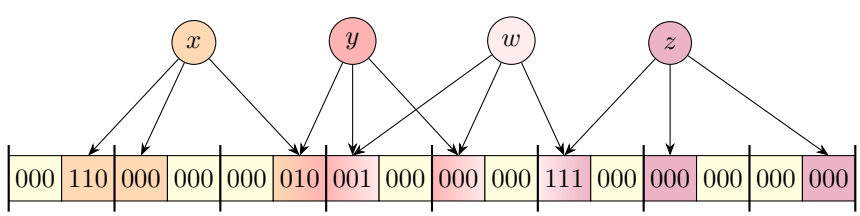

Figure 1: *Binary fuse filter with 16-entry array, 2-entry segments, and 3-bit fingerprints. Vertical bars mark segment boundaries. The filter contains keys* x, y, w *and* z, *with fingerprints* 100, 011, 110 *and* 111, *respectively. Entries hashed to by keys are marked with arrows and the keys' colors. Each key hashes to three consecutive segments.*

To construct a binary fuse filter, choose each array entry such that, for each key, the bitwise-XOR of the three corresponding array entries is equal to the

fingerprint. This requires solving a system of linear equations in $\mathbb{F}_2^m$. Let h_1, h_2 and h_3 be the hash functions, f be the fingerprint function, and A be the array. Each key k yields a linear equation $A[h_1(k)] \oplus A[h_2(k)] \oplus A[h_3(k)] = f(k)$ that must be satisfied (see Figure 2). If some equation is not satisfied, the filter will incorrectly report the corresponding key is *definitely* not in the set. Since false negatives are not permitted, the choice of array entries must satisfy every key's equation.

$$A[1] \oplus A[2] \oplus A[5] = 100 \qquad A[5] \oplus A[6] \oplus A[8] = 011$$

$$A[6] \oplus A[8] \oplus A[10] = 110 \qquad A[10] \oplus A[12] \oplus A[15] = 111$$

Figure 2: *System of linear equations in $\mathbb{F}_2^3$ induced by Figure 1. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). The keys are x, y, w and z with fingerprints 100, 011, 110 and 111, respectively. Array entries and fingerprints are marked with corresponding key color(s). There is one equation for each key.*

To solve the system, model it as a hypergraph. A hypergraph is a graph where edges may contain any subset of vertices. For each entry, create a vertex. For each equation, create an edge containing entries (vertices) in the equation (see Figure 3). Then, 2-peel the hypergraph, that is:

1. Remove vertices of degree zero, then set their entries to 0^m.
2. Remove vertices of degree one and their incident edges. Then, set their entries to the bitwise-XOR of the fingerprint and the two other entries in the edge. If the other entries are not yet set, defer until they are set.
3. Repeat until there are no vertices of degree less than two, or until the empty hypergraph is reached (i.e. all entries set).

If the empty hypergraph is reached, the hypergraph is called 2-peelable. The system is solved if and only if its hypergraph is 2-peelable.

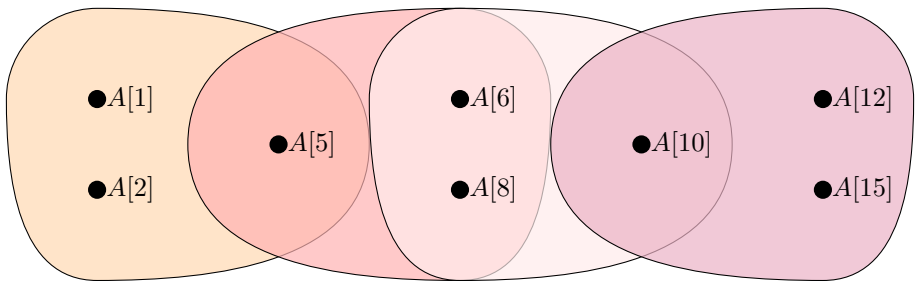

Figure 3: *Hypergraph model of Figure 2. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). Hyperedges correspond to keys x, y, w and z, colored accordingly. Vertices of degree zero not shown.*

In the filter setting, edges represent keys and vertices represent array entries. Higher edge density means lower space usage, since there are more keys (edges) per array entry (vertex). Many families of random hypergraphs exhibit sharp phase transitions in peelability. When the edge density is below a critical value, called the 2-peelability threshold, sampled hypergraphs are almost surely 2-peelable. When the density exceeds this threshold, they are almost surely not. The threshold is therefore a direct measure of space efficiency. A higher threshold means more keys can be stored per array entry.

Binary fuse filters induce the fuse graph family, which achieves the maximum 2-peelability threshold among 3-uniform hypergraph (hypergraphs where each edge contains exactly three vertices) families. This threshold occurs at an edge density of 0.918, or roughly $1.089 \lg \frac{1}{\epsilon}$ bits per key. No family of 3-uniform hypergraphs has higher 2-peelability threshold. More generally, a binary fuse filter with $m \geq 3$ hash functions saturates the 2-peelability threshold for m-uniform hypergraphs.[1]

However, this optimality result is confined to uniform hypergraphs. There may exist families of non-uniform hypergraphs with higher 2-peelability thresholds. If such families also have a low average edge size (i.e. few array accesses per query), filters based on them could potentially reduce memory overhead without degrading query throughput. Therefore, we ask:

1. How can we systematically construct non-uniform hypergraph families with peelability thresholds comparable to or surpassing those of fuse graph families?

2. How can we design efficient approximate membership query data structures by exploiting peelable non-uniform hypergraph families?

[1] In practice, only binary fuse filters with three or four hash functions are relevant. With four hash functions, the 2-peelability threshold is already 0.977, yielding about $1.023 \lg \frac{1}{\epsilon}$ bits per key or within 2.3% of optimal.

2.2 Summary

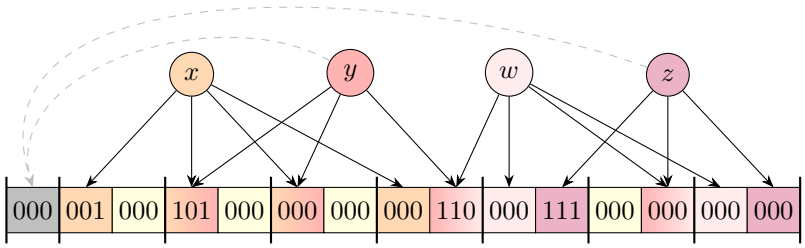

Figure 4: *Non-uniform binary fuse filter with 14-entry array, 2-entry segments, and 3-bit fingerprints. Vertical bars mark segment boundaries. The phantom entry is colored gray. The filter contains keys* x, y, w *and* z, *with fingerprints* 100, 011, 110 *and* 111, *respectively. Entries hashed to by keys are marked with arrows and the keys' colors. Each key hashes to three or four consecutive segments. Dashed gray arrows indicate hashes to the phantom entry.*

$$A[1] \oplus A[3] \oplus A[5] \oplus A[7] = 100$$

$$A[3] \oplus A[5] \oplus A[8] \oplus A[0] = 011$$

$$A[8] \oplus A[9] \oplus A[13] \oplus A[14] = 110$$

$$A[10] \oplus A[13] \oplus A[15] \oplus A[0] = 111$$

Figure 5: *System of linear equations in* $\mathbb{F}_2^3$ *induced by Figure 4. Each* $A[i]$ *is the* i*-th entry in the backing array* A *(zero-indexed). The keys are* x, y, w *and* z *with fingerprints* 100, 011, 110 *and* 111, *respectively. Array entries and fingerprints are marked with corresponding key color(s). The phantom entry is colored gray. There is one equation for each key.*

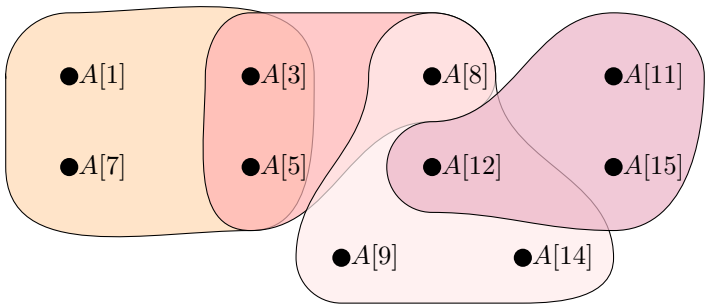

Figure 6: *Hypergraph model of Figure 5. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). Hyperedges correspond to keys x, y, w and z, colored accordingly. Vertices of degree zero and phantom entry $A[0]$ not shown.*

TODO: A 400–500-word summary of the technical contributions of the work, what was accomplished overall, and the limitations of their work.

2.3 Reflection

TODO: A 250-word self reflection of what skills were developed or enhanced in completing those projects.

3 Tuning Secure Learned Bloom Filters

This project was completed for CSC 592: Machine Learning Security.

3.1 Abstract

TODO: A 400–500-word summary of the technical contributions of the work, what was accomplished overall, and the limitations of their work.

3.2 Reflection

TODO: A 250-word self reflection of what skills were developed or enhanced in completing those projects.

4 Entropy-Aware Hashing

This project was completed for CSC 519: Computer Networks.

4.1 Abstract

TODO: A 400–500-word summary of the technical contributions of the work, what was accomplished overall, and the limitations of their work.

4.2 Reflection

TODO: A 250-word self reflection of what skills were developed or enhanced in completing those projects.